



Permisos

Controle qué acciones requieren aprobación para ejecutarse.

OpenCode usa la configuración ``permission`` para decidir si una acción determinada debe ejecutarse automáticamente, avisarle o bloquearse.

A partir de ``v1.1.1``, la configuración booleana heredada ``tools`` está obsoleta y se ha fusionado en ``permission``. La antigua configuración ``tools`` todavía se admite por compatibilidad con versiones anteriores.

Acciones

Cada regla de permiso se resuelve en una de:

``"allow"`` – ejecutar sin aprobación

``"ask"`` – solicitar aprobación

``"deny"`` – bloquea la acción

Configuración

Puede establecer permisos globalmente (con ``*``) y anular herramientas específicas.

opencode.json

```
{
  "$schema": "https://opencode.ai/config.json",
  "permission": {
    "*": "ask",
    "bash": "allow",
    "edit": "deny"
  }
}
```

También puedes configurar todos los permisos a la vez:

opencode.json

```
{
  "$schema": "https://opencode.ai/config.json",
  "permission": "allow"
}
```

Reglas granulares (sintaxis de objeto)

Para la mayoría de los permisos, puede utilizar un objeto para aplicar diferentes acciones según la entrada de la herramienta.

opencode.json

```
{
  "$schema": "https://opencode.ai/config.json",
  "permission": {
    "bash": {
      "*": "ask",
      "git *": "allow",
      "npm *": "allow",
      "rm *": "deny",
      "grep *": "allow"
    },
    "edit": {
      "*": "deny",
      "packages/web/src/content/docs/*.mdx": "allow"
    }
  }
}
```

Las reglas se evalúan según la coincidencia de patrones, y la **última regla coincidente gana**. Un patrón común es poner primero la regla general ``*`` y después reglas más específicas.

Comodines

Los patrones de permisos utilizan una simple coincidencia de comodines:

``*`` coincide con cero o más de cualquier carácter

``?`` coincide exactamente con un carácter

Todos los demás caracteres coinciden literalmente

Expansión del directorio de inicio

Puede usar ``~`` o ``$HOME`` al comienzo de un patrón para hacer referencia a su directorio de inicio. Esto es particularmente útil para las reglas ``external_directory``.

```
`~/projects/*` -> `/Users/username/projects/*`  
  
`$HOME/projects/*` -> `/Users/username/projects/*`  
  
`~` -> `/Users/username`
```

Directorios externos

Utilice ``external_directory`` para permitir llamadas a herramientas que toquen rutas fuera del directorio de trabajo donde se inició OpenCode. Esto se aplica a cualquier herramienta que tome una ruta como entrada (por ejemplo, ``read``, ``edit``, ``glob``, ``grep`` y muchos comandos ``bash``).

La expansión del hogar (como ``~/...``) solo afecta la forma en que se escribe un patrón. No hace que una ruta externa forme parte del espacio de trabajo actual, por lo que las rutas fuera del directorio de trabajo aún deben permitirse a través de ``external_directory``.

Por ejemplo, esto permite el acceso a todo lo que se encuentra en ``~/projects/personal/``:

opencode.json

```
{  
  "$schema": "https://opencode.ai/config.json",  
  "permission": {  
    "external_directory": {  
      "~/projects/personal/**": "allow"  
    }  
  }  
}
```

Cualquier directorio permitido aquí hereda los mismos valores predeterminados que el espacio de trabajo actual. Dado que ``read`` tiene por defecto ``allow``, también se permiten lecturas para entradas bajo ``external_directory`` a menos que se anulen. Agregue reglas explícitas cuando una herramienta deba restringirse en estas rutas, como bloquear ediciones mientras se mantienen las lecturas:

```
opencode.json

{
  "$schema": "https://opencode.ai/config.json",
  "permission": {
    "external_directory": {
      "~/projects/personal/**": "allow"
    },
    "edit": {
      "~/projects/personal/**": "deny"
    }
  }
}
```

Mantenga la lista centrada en rutas confiables y aplique reglas adicionales de permitir o denegar según sea necesario para otras herramientas (por ejemplo, ``bash``).

Permisos disponibles

Los permisos OpenCode están codificados por el nombre de la herramienta, además de un par de medidas de seguridad:

- ``read`` – leer un archivo (coincide con la ruta del archivo)
- ``edit`` – todas las modificaciones de archivos (cubre ``edit``, ``write``, ``patch``)
- ``glob`` – globalización de archivos (coincide con el patrón global)
- ``grep`` – búsqueda de contenido (coincide con el patrón de expresiones regulares)
- ``bash``: ejecuta comandos de shell (coincide con comandos analizados como ``git status --porcelain``)
- ``task`` – lanzamiento de subagentes (coincide con el tipo de subagente)
- ``skill`` – cargar una habilidad (coincide con el nombre de la habilidad)
- ``lsp``: ejecución de consultas LSP (actualmente no granulares)
- ``webfetch`` – obteniendo una URL (coincide con la URL)

``websearch`` – búsqueda web (coincide con la consulta)

``external_directory``: se activa cuando una herramienta toca rutas fuera del directorio de trabajo del proyecto.

``doom_loop``: se activa cuando la misma llamada de herramienta se repite 3 veces con entrada idéntica

Valores predeterminados

Si no especifica nada, OpenCode comienza desde valores predeterminados permisivos:

La mayoría de los permisos están predeterminados en ``"allow"``.

``doom_loop`` y ``external_directory`` por defecto son ``"ask"``.

``read`` es ``"allow"``, pero los archivos ``.env`` están denegados de forma predeterminada:

opencode.json
<pre>{ "permission": { "read": { "*": "allow", "*.env": "deny", "*.env.*": "deny", "*.env.example": "allow" } } }</pre>

¿Qué significa “preguntar”?

Cuando OpenCode solicita aprobación, la interfaz de usuario ofrece tres resultados:

``once`` – aprobar solo esta solicitud

``always``: aprueba solicitudes futuras que coincidan con los patrones sugeridos (para el resto de la sesión actual OpenCode)

``reject`` – rechazar la solicitud

La herramienta proporciona el conjunto de patrones que ``always`` aprobaría (por ejemplo, las aprobaciones de bash generalmente incluyen en la lista blanca un prefijo de comando seguro como ``git status*``).

Agentes

Puede anular los permisos por agente. Los permisos del agente se combinan con la configuración global y las reglas del agente tienen prioridad. [Más información](#) sobre los permisos de los agentes.

NOTA

Consulte la sección [Reglas granulares \(sintaxis de objeto\)](#) anterior para obtener ejemplos de coincidencia de patrones más detallados.

opencode.json

```
{
  "$schema": "https://opencode.ai/config.json",
  "permission": {
    "bash": {
      "*": "ask",
      "git *": "allow",
      "git commit *": "deny",
      "git push *": "deny",
      "grep *": "allow"
    }
  },
  "agent": {
    "build": {
      "permission": {
        "bash": {
          "*": "ask",
          "git *": "allow",
          "git commit *": "ask",
          "git push *": "deny",
          "grep *": "allow"
        }
      }
    }
  }
}
```

También puede configurar los permisos del agente en Markdown:

~/.config/opencode/agents/review.md

description: Code review without edits

mode: subagent

permission:

edit: deny

bash: ask

webfetch: deny

Only analyze code and suggest changes.

🔗 CONSEJO

Utilice la coincidencia de patrones para comandos con argumentos. ``"grep *"`` permite ``grep pattern file.txt``, mientras que ``"grep"`` solo lo bloquearía. Los comandos como ``git status`` funcionan para el comportamiento predeterminado pero requieren permiso explícito (como ``"git status *"``) cuando se pasan argumentos.

✎ Edita esta página

© Anomaly

🐛 Found a bug? Open an issue

Última actualización: 21 may 2026

🗨 Join our Discord community

🇪🇸 Español

